Relatório – Trabalho Prático 1

Jogo da Forca

Unidade Curricular: Sistemas Distribuídos

Trabalho Realizado Por:

Diogo Baltazar (52264)
Davide Cravo (52868)

Universidade da Beira Interior
19 de abril de 2026

Conteúdo

1 Introdução **3**

2 Objetivo do Projeto **3**

3 Explicação Geral do Jogo **4**

4 Tecnologias Utilizadas e Justificação **4**
4.1 Java 4
4.2 TCP/IP 4
4.3 Threads 5
4.4 Ficheiros de Texto 5
4.5 Terminal / Consola 5

5 Arquitetura do Projeto **5**
5.1 Visão Geral 5
5.2 Servidor 5
5.3 Cliente 6
5.4 ClienteHandler 6

6 Funcionamento do Jogo no Nosso Sistema **6**
6.1 Entrada no Lobby 6
6.2 Início do Jogo 7
6.3 Funcionamento das Rondas 7
6.4 Tratamento das Jogadas 7
6.5 Exemplo de Revelação Parcial 7
6.6 Representação Visual da Forca 8
6.7 Fim do Jogo 9

7 Comunicação Cliente-Servidor **9**
7.1 Mensagens enviadas pelo servidor 9
7.2 Mensagem enviada pelo cliente 9

8 Código Relevante – Comunicação **9**
8.1 Criação do servidor TCP 9
8.2 Aceitação de clientes 10
8.3 Envio de mensagens 10
8.4 Timeout de leitura 10

9 Sincronização e Rondas **10**

10 Funcionalidades Diferenciadoras **10**

11 Ficheiro Externo de Palavras **11**

12 Controlo de Vitória e Derrota **11**
12.1 Vitória 11
12.2 Derrota 11

13 Organização do Código **11**

14 Dificuldades Encontradas **12**

15 Vídeo de Demonstração **12**

16 Repositório GitHub **12**

17 Conclusão **13**

1. Introdução

O presente trabalho teve como objetivo o desenvolvimento de uma aplicação baseada no clássico **Jogo da Forca**, adaptado para um ambiente **multijogador síncrono**, onde entre 2 e 4 jogadores participam em simultâneo através de uma arquitetura **cliente-servidor**.

A aplicação foi desenvolvida em Java, utilizando sockets TCP/IP para a comunicação entre cliente e servidor. O servidor centraliza toda a lógica do jogo, enquanto cada cliente apenas apresenta a informação ao utilizador e envia a respetiva jogada.

Ao longo do desenvolvimento procurou-se, não só cumprir os requisitos pedidos no enunciado, mas também tornar o jogo mais claro e agradável de utilizar. Por essa razão, foram incluídas algumas melhorias, como a representação visual da forca em ASCII, a revelação parcial de letras corretas em certas tentativas erradas e a utilização de um ficheiro externo com 100 palavras.

2. Objetivo do Projeto

O objetivo principal deste trabalho foi implementar um sistema distribuído funcional onde vários jogadores pudessem participar no mesmo jogo da forca em tempo real, através de uma rede TCP/IP.

Com este projeto foi possível aplicar na prática vários conceitos da unidade curricular, nomeadamente:

- arquitetura cliente-servidor;
- comunicação em rede;
- sincronização entre vários participantes;
- gestão de estado partilhado;
- controlo de múltiplos clientes em simultâneo.

Para além disso, procurou-se também melhorar a experiência de utilização, tornando o jogo mais intuitivo e visual. Entre as melhorias realizadas destacam-se:

- utilização de uma forca visual em ASCII em vez de mostrar apenas o número de tentativas;
- revelação parcial de letras corretas quando um jogador tenta uma palavra errada mas com letras bem colocadas;
- utilização de um ficheiro externo com 100 palavras;
- separação clara entre responsabilidades do servidor, cliente e `ClienteHandler`.

3. Explicação Geral do Jogo

No nosso sistema, todos os jogadores participam na mesma jogada e tentam descobrir uma palavra comum escolhida pelo servidor.

A palavra correta, a máscara apresentada, as letras já utilizadas e o número de tentativas restantes são elementos **partilhados por todos os jogadores**. Isto significa que qualquer jogada correta beneficia todos os participantes, uma vez que novas letras passam a ser reveladas para todos.

Desta forma, existe uma componente colaborativa, porque todos contribuem para descobrir progressivamente a palavra.

No entanto, a vitória final é individual, ou partilhada em caso de empate na mesma ronda, sendo vencedor o jogador que acertar a palavra completa, de acordo com as regras definidas no enunciado.

Assim, o jogo combina:

- **cooperação**, através do progresso comum da palavra;
- **competição**, através da tentativa de acertar a palavra completa antes dos restantes jogadores.

4. Tecnologias Utilizadas e Justificação

4.1 Java

A linguagem Java foi escolhida por disponibilizar bibliotecas robustas para programação concorrente e comunicação em rede.

Vantagens:

- biblioteca nativa `java.net`;
- facilidade na criação de sockets;
- programação orientada a objetos;
- portabilidade entre sistemas operativos;
- gestão automática de memória;
- boa organização do código em classes.

4.2 TCP/IP

Foi utilizado o protocolo TCP, uma vez que garante:

- entrega fiável das mensagens;
- ordem correta de receção;
- ligação persistente entre cliente e servidor.

Estas características são essenciais num jogo síncrono onde mensagens perdidas ou fora de ordem poderiam comprometer o estado do jogo.

4.3 Threads

Foram utilizadas threads para permitir múltiplos clientes ligados ao mesmo tempo. Embora a lógica do jogo esteja concentrada no servidor, cada ligação é tratada individualmente, permitindo comunicação simultânea com vários jogadores.

4.4 Ficheiros de Texto

Foi utilizado um ficheiro externo denominado `palavras`, contendo 100 palavras possíveis.

Vantagens:

- fácil manutenção;
- alteração sem recompilar;
- separação entre lógica e conteúdo;
- maior flexibilidade para atualizar o jogo.

4.5 Terminal / Consola

A interface do sistema foi implementada em modo texto, utilizando a consola. Esta opção foi escolhida por ser simples, leve e adequada ao contexto do trabalho, permitindo focar o desenvolvimento nos conceitos distribuídos e não numa interface gráfica.

5. Arquitetura do Projeto

O sistema segue uma arquitetura **Cliente-Servidor**.

5.1 Visão Geral

A lógica do jogo encontra-se centralizada no servidor. Os clientes apenas enviam jogadas e apresentam ao utilizador a informação recebida.

Desta forma, evita-se que cada cliente mantenha uma versão própria do estado do jogo, reduzindo inconsistências e garantindo que todos os jogadores veem o mesmo progresso.

5.2 Servidor

O servidor é o componente principal do sistema e é responsável por:

- aceitar ligações de clientes;
- gerir o lobby inicial;
- selecionar aleatoriamente a palavra do jogo;
- controlar o número de tentativas disponíveis;
- enviar o estado atual do jogo aos clientes;
- recolher as jogadas de todos os jogadores;
- processar a lógica de cada ronda;

- verificar condições de vitória e derrota;
- encerrar as ligações no final.

5.3 Cliente

Cada cliente representa um jogador e é responsável por:

- estabelecer ligação ao servidor;
- receber mensagens e estados do jogo;
- mostrar a informação ao utilizador;
- pedir uma jogada ao jogador;
- enviar essa jogada ao servidor.

O cliente não toma decisões sobre a lógica do jogo. Toda a decisão sobre vitória, derrota, atualização da máscara e controlo das rondas é feita pelo servidor.

5.4 ClienteHandler

O `ClienteHandler` atua como intermediário entre o servidor e cada socket individual.

Funções:

- ler mensagens do cliente;
- aplicar timeout na leitura;
- enviar mensagens ao jogador;
- encerrar a ligação individual.

Esta separação torna o projeto mais organizado e facilita a manutenção do código.

6. Funcionamento do Jogo no Nosso Sistema

No nosso sistema, o jogo funciona segundo uma sequência bem definida, controlada integralmente pelo servidor.

6.1 Entrada no Lobby

Quando o servidor é iniciado, fica à espera de ligações de clientes. O primeiro cliente que entra inicia o período de lobby. Durante esse período, o servidor aceita novos jogadores até um máximo de 4.

Cada jogador que entra recebe uma mensagem de boas-vindas com o seu identificador e com o número de jogadores já ligados:

```
WELCOME <id> <players_total>
```

Se o número máximo de jogadores já tiver sido atingido, ou se o jogo já tiver começado, o servidor responde com:

```
FULL
```

Esta fase é importante porque controla a entrada dos jogadores e garante que o jogo só começa quando existem participantes suficientes.

6.2 Início do Jogo

Quando estão reunidas as condições para começar, o servidor escolhe aleatoriamente uma palavra do ficheiro `palavras`. Em seguida, cria a máscara inicial da palavra, substituindo todas as letras por sublinhados.

Depois disso, envia a todos os clientes a mensagem de início do jogo:

```
START <mask> <attempts> <round_timeout_ms>
```

Por exemplo, se a palavra tiver 5 letras e o jogo começar com 10 tentativas, a mensagem poderá indicar:

```
START _____ 10 25000
```

Assim, todos os jogadores iniciam a partida com a mesma informação.

6.3 Funcionamento das Rondas

O jogo decorre por rondas síncronas. Em cada ronda, o servidor envia a todos os jogadores o estado atual do jogo:

```
ROUND <k> <mask> <attempts> <used_letters>
```

Cada jogador pode então introduzir uma letra ou tentar a palavra completa. O cliente envia sempre a jogada no formato:

```
GUESS <texto>
```

O servidor recolhe todas as jogadas antes de atualizar o estado global. Isto significa que nenhum jogador beneficia de ver primeiro o efeito da jogada dos outros, o que torna o jogo mais justo.

6.4 Tratamento das Jogadas

No nosso sistema existem três casos principais:

1. **Jogada com uma letra:** se a letra existir na palavra, a máscara é atualizada. Caso contrário, perde-se uma tentativa.

2. **Jogada com a palavra completa correta:** o jogador é declarado vencedor.

3. **Jogada com a palavra completa incorreta:** se existirem letras corretas nas posições corretas, essas letras são reveladas na máscara; se não houver qualquer progresso, perde-se uma tentativa.

Esta terceira situação é uma das funcionalidades diferenciadoras do projeto, porque torna o jogo mais interessante e recompensa tentativas parcialmente corretas.

6.5 Exemplo de Revelação Parcial

Supondo que a palavra correta é `PORTA`, se um jogador tentar `MORTA`, a palavra não está totalmente correta, mas existem letras bem posicionadas.

Assim, a máscara passa a mostrar:

```
_ORTA
```

Código utilizado:

```
if (letraTentativa == letraCorreta &&
    novaMascara.charAt(i) == '_') {
    novaMascara.setCharAt(i, letraCorreta);
}
```

6.6 Representação Visual da Forca

Em vez de apresentar apenas o número de tentativas restantes, foi criada uma representação gráfica em ASCII.

Cada erro acrescenta uma nova parte da forca ou do boneco:

1. Base
2. Poste
3. Barra superior
4. Corda
5. Cabeça
6. Tronco
7. Braço esquerdo
8. Braço direito
9. Perna esquerda
10. Perna direita

Código ilustrativo:

```
if (erros >= 1) g[6][0] = '=';
if (erros >= 2) g[1][0] = '|';
if (erros >= 3) g[0][1] = '-';
if (erros >= 4) g[1][4] = '|';
if (erros >= 5) g[2][4] = 'O';
if (erros >= 6) g[3][4] = '|';
if (erros >= 7) g[3][3] = '/';
if (erros >= 8) g[3][5] = '\\';
if (erros >= 9) g[4][3] = '/';
if (erros >= 10) g[4][5] = '\\';
```

Exemplo final:

```
+---+
|   |
|   O
|  /|\
|  / \
|
=====
```

Esta opção torna o jogo mais apelativo visualmente e diferencia o projeto de uma implementação mínima.

6.7 Fim do Jogo

O jogo termina quando se verifica uma das seguintes situações:

- um ou mais jogadores acertam a palavra completa na mesma ronda;
- o número de tentativas chega a zero.

Em caso de vitória, o servidor envia:

```
END WIN <ids> <word>
```

Em caso de derrota, o servidor envia:

```
END LOSE <word>
```

Depois disso, todas as ligações são encerradas.

7. Comunicação Cliente-Servidor

7.1 Mensagens enviadas pelo servidor

- `WELCOME <id> <players_total>`
- `START <mask> <attempts> <round_timeout_ms>`
- `ROUND <k> <mask> <attempts> <used_letters>`
- `STATE <mask> <attempts> <used_letters>`
- `END WIN <ids> <word>`
- `END LOSE <word>`
- `FULL`

7.2 Mensagem enviada pelo cliente

- `GUESS <texto>`

O uso de mensagens simples em texto permitiu implementar um protocolo leve, fácil de depurar e suficientemente claro para este tipo de aplicação.

8. Código Relevante – Comunicação

8.1 Criação do servidor TCP

```
servidorSocket = new ServerSocket(PORTA);
```

Este código cria o socket principal do servidor e coloca-o à espera de novas ligações.

8.2 Aceitação de clientes

```
Socket socket = servidorSocket.accept();
ClienteHandler jogador = new ClienteHandler(socket);
```

Sempre que um cliente se liga, o servidor aceita essa ligação e cria um handler dedicado para a comunicação com esse jogador.

8.3 Envio de mensagens

```
jogador.enviarMensagem("WELCOME " + jogador.getId() + " " +
   jogadores.size());
```

As mensagens são enviadas de forma textual, simplificando o protocolo e a depuração do sistema.

8.4 Timeout de leitura

```
socket.setSoTimeout(timeoutMs);
String linha = in.readLine();
```

Caso o jogador não responda dentro do tempo definido, a jogada é considerada vazia e essa ronda é processada normalmente.

9. Sincronização e Rondas

Uma parte importante do projeto foi garantir que o jogo decorre de forma síncrona.

Em cada ronda:

1. o servidor envia o estado atual;
2. todos os jogadores respondem;
3. o servidor recolhe as jogadas;
4. apenas depois disso atualiza o estado global.

Esta abordagem foi importante para garantir consistência no estado partilhado. Se o servidor atualizasse a palavra assim que recebesse a jogada de apenas um jogador, os restantes poderiam jogar já com informação diferente, o que tornaria o sistema injusto.

Desta forma, todos os jogadores participam em condições idênticas e observam a mesma evolução do jogo em cada ronda.

10. Funcionalidades Diferenciadoras

As funcionalidades que mais distinguem o nosso trabalho de uma implementação mínima são:

- revelação parcial de letras corretas em palavras erradas;
- forca visual em ASCII com evolução progressiva;

- utilização de ficheiro externo com 100 palavras;
- interface textual mais intuitiva;
- interpretação diferenciada do resultado final para cada jogador.

Estas decisões permitiram não só cumprir o enunciado, mas também melhorar a experiência de utilização e tornar o projeto mais completo.

11. Ficheiro Externo de Palavras

As palavras possíveis foram colocadas num ficheiro externo denominado `palavras`, contendo 100 palavras.

Código:

```
BufferedReader br =
new BufferedReader(new FileReader("palavras"));
```

Vantagens:

- facilidade de manutenção;
- possibilidade de alterar palavras sem recompilar;
- cumprimento do requisito de lista fixa.

12. Controlo de Vitória e Derrota

12.1 Vitória

Se um ou mais jogadores acertarem a palavra completa na mesma ronda:

```
if (tentativa.equals(palavraEscolhida)) {
    vencedores.add(jogador.getId());
}
```

12.2 Derrota

Quando as tentativas chegam a zero:

```
if (tentativas <= 0) {
    terminarJogoComDerrota();
}
```

13. Organização do Código

O projeto foi dividido em três classes principais:

- `Servidor.java`
- `Cliente.java`

- `ClienteHandler.java`

Esta organização permitiu separar melhor responsabilidades e tornar o projeto mais fácil de compreender e manter.

14. Dificuldades Encontradas

Durante o desenvolvimento destacam-se os seguintes desafios:

- sincronização correta das rondas;
- gestão de múltiplos clientes;
- implementação de timeout por jogador;
- manutenção de estado consistente;
- construção de uma interface textual intuitiva;
- adaptação do protocolo para refletir corretamente os vencedores.

Estas dificuldades foram importantes para consolidar conhecimentos da unidade curricular, uma vez que obrigaram a pensar não apenas na lógica do jogo, mas também na comunicação e consistência do sistema como um todo.

15. Vídeo de Demonstração

Para complementar o relatório, foi produzido um vídeo de demonstração do funcionamento do sistema, mostrando o arranque do servidor, ligação dos clientes, progresso do jogo e condições de vitória e derrota.

Link do vídeo no YouTube:

https://youtu.be/D0jB69IjPfo

16. Repositório GitHub

De forma a facilitar a consulta do código-fonte, organização do projeto e respetiva execução, todo o trabalho desenvolvido foi disponibilizado num repositório GitHub.

O repositório contém:

- código-fonte completo do servidor;
- código-fonte completo do cliente;
- classe auxiliar `ClienteHandler`;
- ficheiro externo `palavras`;
- ficheiro `README.md` com instruções de utilização;
- histórico de versões e organização do projeto.

Link do repositório:

https://github.com/DavideDC32/JogoForca

17. Conclusão

Este trabalho permitiu aplicar conceitos fundamentais de Sistemas Distribuídos, nomeadamente:

- arquitetura cliente-servidor;
- comunicação em rede;
- concorrência;
- sincronização.

O resultado final foi uma aplicação funcional, estável e visualmente melhorada, destacando-se a forca gráfica, a revelação parcial de palavras e a utilização de um ficheiro externo para gerir as palavras do jogo.

Para além de cumprir os requisitos principais do enunciado, o projeto permitiu perceber de forma prática a importância de centralizar a lógica no servidor, controlar a sincronização entre jogadores e manter um estado consistente ao longo de todo o jogo.

De uma forma geral, consideramos que o projeto cumpriu os objetivos definidos e representou uma boa aplicação prática dos conteúdos abordados na unidade curricular.